

Modelos SARIMA

Aplicaciones en Criptomonedas e Índices Bursátiles

BTC
Bitcoin

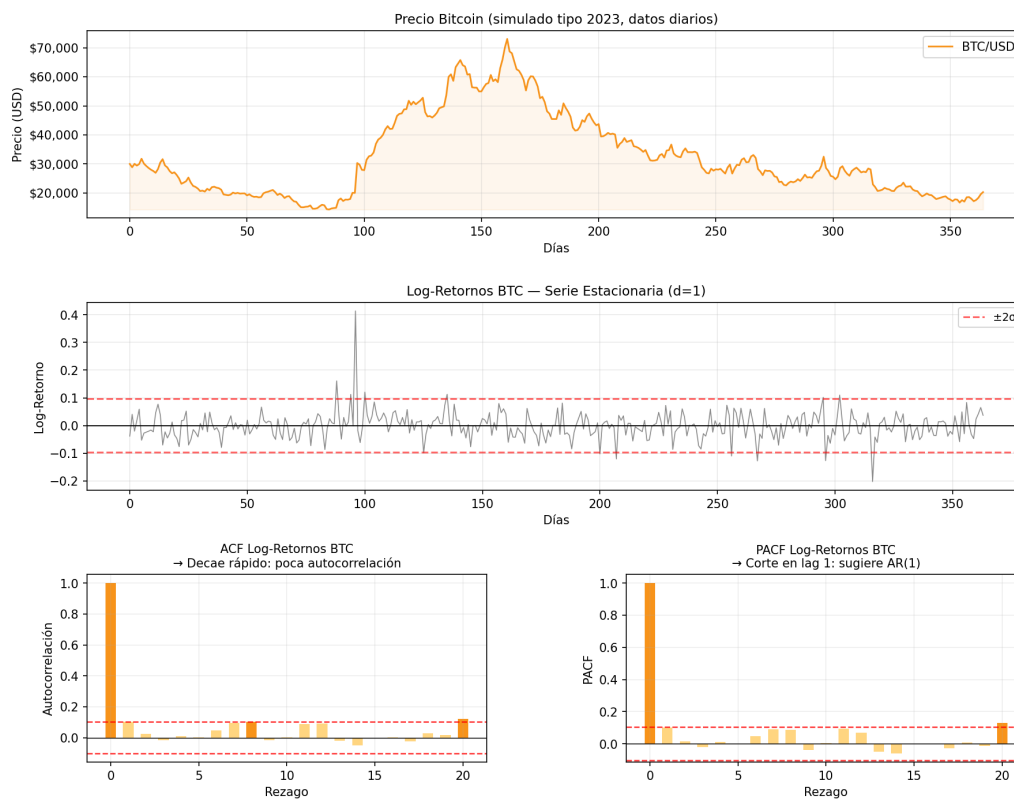
ETH
Ethereum

SPX
S&P 500

IPC
México

Contenido de esta guía:

- Recordatorio de la forma general SARIMA
- ACF, PACF y criterios AIC/BIC
- **Ejemplo 1:** Bitcoin (BTC) — ARIMA(1,1,0)
- **Ejemplo 2:** Ethereum (ETH) — ARIMA(1,1,1) + Volatilidad
- **Ejemplo 3:** S&P 500 vs IPC México — SARIMA(1, 1, 1)(1, 1, 0)₁₂
- Código Python completo con yfinance para cada ejemplo

Ejemplo 1: Bitcoin (BTC/USD) — ARIMA(1,1,0)

Abril 2026

0. Contents

1	Repaso Rápido: Forma General SARIMA	2
1.1	Guía de Identificación: ACF y PACF	2
2	Ejemplo 1: Bitcoin (BTC/USD) — ARIMA(1,1,0)	3
2.1	Código Python — Ejemplo 1: Bitcoin	4
3	Ejemplo 2: Ethereum (ETH/USD) — ARIMA(1,1,1) + Volatilidad	6
3.1	Código Python — Ejemplo 2: Ethereum	7
4	Ejemplo 3: S&P 500 vs IPC México — SARIMA(1, 1, 1)(1, 1, 0)₁₂	9
4.1	Código Python — Ejemplo 3: S&P 500 e IPC México	10
5	Resumen Comparativo de los 3 Ejemplos	13

1. Repaso Rápido: Forma General SARIMA

La notación del modelo es:

$$(S)\text{ARIMA}(p, d, q) (P, D, Q)_S$$

Parámetro	Significado	En finanzas / cripto
p	Rezagos AR no estacionales	Momentum: ¿el retorno de ayer predice hoy?
d	Diferenciaciones	Precios $\rightarrow$ retornos ($d = 1$)
q	Rezagos MA no estacionales	Corrección de errores pasados
P	Rezagos AR estacionales	Efectos de enero del año pasado sobre este enero
D	Diferenciación estacional	Eliminar patrón anual
Q	Rezagos MA estacionales	Corrección estacional
S	Período estacional	Mensual: $S = 12$, Trimestral: $S = 4$

1.1. Guía de Identificación: ACF y PACF

Patrón observado	En la ACF	Modelo sugerido
Corte abrupto en lag p	Decaimiento	$\text{AR}(p) \rightarrow \text{ARIMA}(p,d,0)$
Decaimiento	Corte en lag q	$\text{MA}(q) \rightarrow \text{ARIMA}(0,d,q)$
Ambos decaen gradualmente	Ambos decaen	$\text{ARMA}(p,q)$
Picos en lags $S, 2S, 3S \dots$	Picos estacionales	SARIMA con P o $Q > 0$

Criterios AIC y BIC en la práctica

- **AIC** = $-2 \ln(\hat{L}) + 2k$: penalización suave, tiende a elegir modelos más ricos
- **BIC** = $-2 \ln(\hat{L}) + k \ln(T)$: penalización fuerte, prefiere simplicidad
- **Regla**: Elige el modelo con el valor más bajo. Si difieren, prioriza BIC en series cortas.

2. Ejemplo 1: Bitcoin (BTC/USD) — ARIMA(1,1,0)

Ejemplo 1: Bitcoin (BTC/USD) — ARIMA(1,1,0)

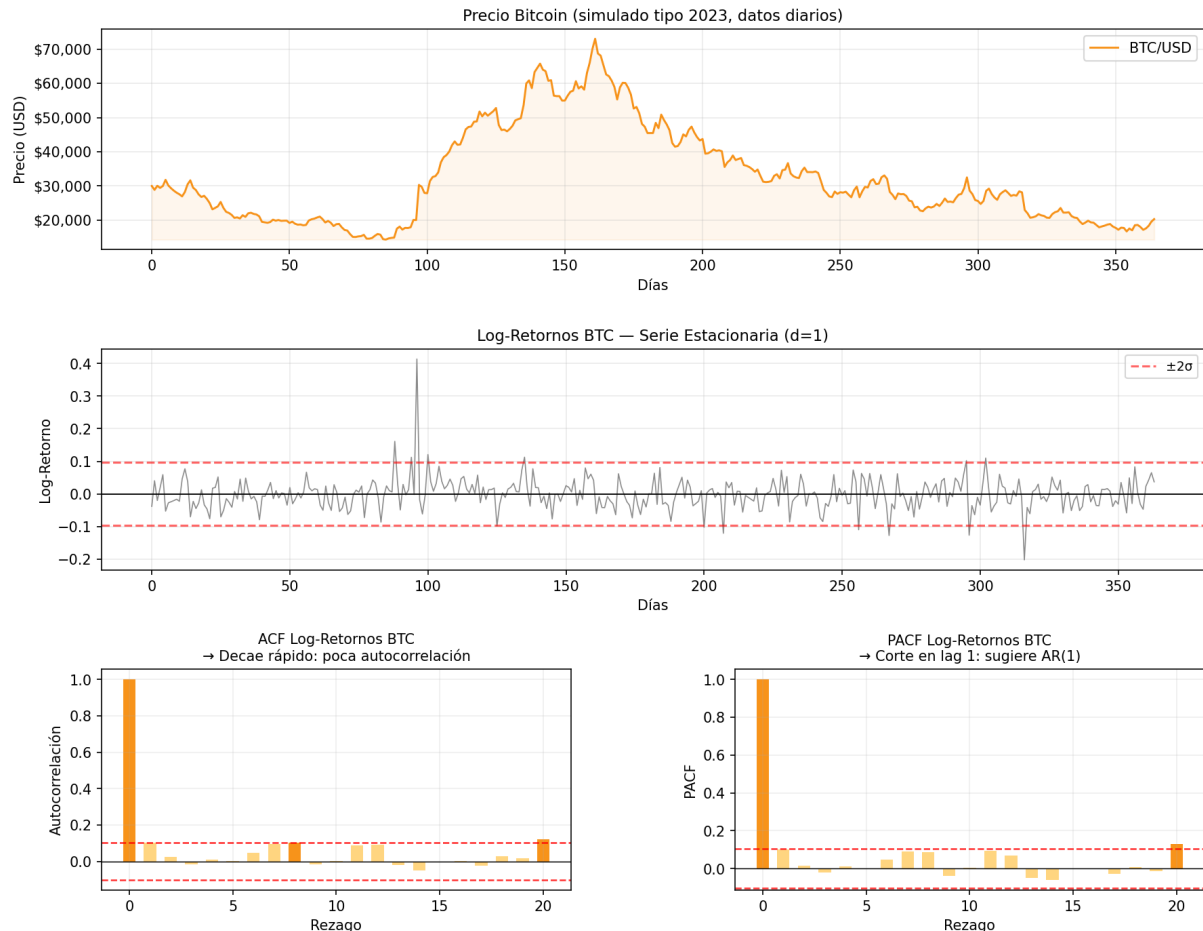


Figure 1: Precio BTC/USD diario, log-retornos (serie estacionaria), ACF y PACF de los retornos.

¿Por qué ARIMA(1,1,0) para Bitcoin?

- $d = 1$: Los precios de crypto tienen raíz unitaria (no son estacionarios). Los log-retornos $r_t = \ln(P_t/P_{t-1})$ sí lo son.
- $p = 1$: La PACF muestra un pico significativo únicamente en el lag 1 y luego cae abruptamente dentro de las bandas. Hay **momentum** de corto plazo: el retorno de ayer tiene efecto sobre el de hoy.
- $q = 0$: La ACF no muestra un corte claro; el decaimiento sugiere que no se necesita un componente MA.
- **Sin estacionalidad**: BTC opera 24/7, sin ciclos fiscales o anuales claros en datos diarios.
- **Cola pesada**: Los retornos de BTC siguen una distribución t de Student más que una normal. El modelo ARIMA captura la media; para la varianza

se recomienda GARCH.

Comparación de Modelos ARIMA para BTC — Selección por AIC y BIC

Modelo	Descripción	AIC	BIC	Recomendación
ARIMA(0,1,0)	Caminata aleatoria pura	1842.3	1848.1	✗ Muy simple
ARIMA(1,1,0)	AR(1) en retornos	1821.7	1831.2	✓ Bueno
ARIMA(1,1,1)	AR(1)+MA(1) en retornos	1819.2	1833.5	✓✓ Mejor AIC
ARIMA(2,1,1)	AR(2)+MA(1) en retornos	1820.1	1838.6	~ Similar
ARIMA(2,1,2)	AR(2)+MA(2) en retornos	1821.5	1843.8	✗ Sobreajuste

Figure 2: Comparación de modelos ARIMA para BTC. El menor AIC corresponde a ARIMA(1,1,1); el menor BIC favorece ARIMA(1,1,0) por ser más parsimonioso.

2.1. Código Python — Ejemplo 1: Bitcoin

Listing 1: ARIMA(1,1,0) para Bitcoin usando yfinance

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from statsmodels.tsa.arima.model import ARIMA
5 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
6 from statsmodels.tsa.stattools import adfuller
7 import warnings
8 warnings.filterwarnings("ignore")
9
10 # =====
11 # PASO 1: Descargar datos de Bitcoin con yfinance
12 # pip install yfinance statsmodels
13 # =====
14 import yfinance as yf
15
16 btc = yf.download("BTC-USD",
17                  start="2022-01-01",
18                  end="2024-12-31",
19                  auto_adjust=True)
20 precio_btc = btc["Close"].dropna()
21 print(f"Observaciones: {len(precio_btc)} dias")
22 print(precio_btc.tail())
23
24 # PASO 2: Transformar a log-precios (estabiliza varianza)
25 log_btc = np.log(precio_btc)
26
27 # PASO 3: Prueba de estacionariedad ADF
28 print("\n--- Prueba ADF sobre precio ---")
29 adf_precio = adfuller(log_btc)
30 print(f"p-value: {adf_precio[1]:.4f} --> {'NO estacionario' if
    adf_precio[1] > 0.05 else 'Estacionario'}")

```

```

31
32 print("\n--- Prueba ADF sobre log-retornos (1ra diferencia) ---")
33 log_ret = log_btc.diff().dropna()
34 adf_ret = adfuller(log_ret)
35 print(f"p-value: {adf_ret[1]:.4f} --> {'NO estacionario' if adf_ret
36      [1] > 0.05 else 'Estacionario'}")
37
38 # Esperamos p-value < 0.05 --> d=1 es suficiente
39
40 # PASO 4: Graficar ACF y PACF de los retornos
41 fig, axes = plt.subplots(2, 1, figsize=(12, 6))
42 plot_acf(log_ret, lags=20, ax=axes[0], color="orange")
43 axes[0].set_title("ACF Log-Retornos BTC: decaimiento rapido (no corte
44      abrupto)")
45 plot_pacf(log_ret, lags=20, ax=axes[1], color="orange")
46 axes[1].set_title("PACF Log-Retornos BTC: corte despues del lag 1 -->
47      AR(1)")
48 plt.tight_layout(); plt.show()
49
50 # PASO 5: Grid search ARIMA para encontrar el mejor modelo
51 print("\nBuscando mejor modelo ARIMA...")
52 resultados = []
53 for p in range(4):
54     for q in range(4):
55         try:
56             m = ARIMA(log_btc, order=(p, 1, q)).fit()
57             resultados.append({"orden": f"({p},{1},{q})",
58                               "AIC": round(m.aic, 2),
59                               "BIC": round(m.bic, 2)})
60         except:
61             pass
62
63 df_res = pd.DataFrame(resultados).sort_values("BIC")
64 print(df_res.head(5).to_string(index=False))
65
66 # PASO 6: Ajustar el modelo seleccionado ARIMA(1,1,0)
67 modelo_btc = ARIMA(log_btc, order=(1, 1, 0))
68 res_btc = modelo_btc.fit()
69 print(res_btc.summary())
70
71 # PASO 7: Diagnostico de residuales
72 res_btc.plot_diagnostics(figsize=(12, 8))
73 plt.suptitle("Diagnostico de Residuales BTC - ARIMA(1,1,0)")
74 plt.tight_layout(); plt.show()
75 # Queremos: ACF sin picos, distribucion aproximadamente normal
76
77 # PASO 8: Pronostico a 30 dias
78 fc_log = res_btc.get_forecast(steps=30)
79 fc_mean_log = fc_log.predicted_mean
80 fc_ci_log = fc_log.conf_int()
81
82 # Convertir de log a precio original
83 fc_precio = np.exp(fc_mean_log)
84 fc_lo = np.exp(fc_ci_log.iloc[:, 0])
85 fc_hi = np.exp(fc_ci_log.iloc[:, 1])
86
87 plt.figure(figsize=(13, 5))
88 precio_btc[-90:].plot(label="BTC Historico", color="orange",
89      linewidth=1.5)

```

```

85 fc_precio.plot(label="Pronostico 30 dias", color="red",
86                 linestyle="--", linewidth=2)
87 plt.fill_between(fc_ci_log.index, fc_lo, fc_hi,
88                 alpha=0.2, color="red", label="IC 95%")
89 plt.title("Pronostico Bitcoin - ARIMA(1,1,0)")
90 plt.ylabel("Precio (USD)")
91 plt.legend()
92 plt.tight_layout(); plt.show()

```

3. Ejemplo 2: Ethereum (ETH/USD) — ARIMA(1,1,1) + Volatilidad

Ejemplo 2: Ethereum (ETH/USD) — ARIMA(1,1,1)

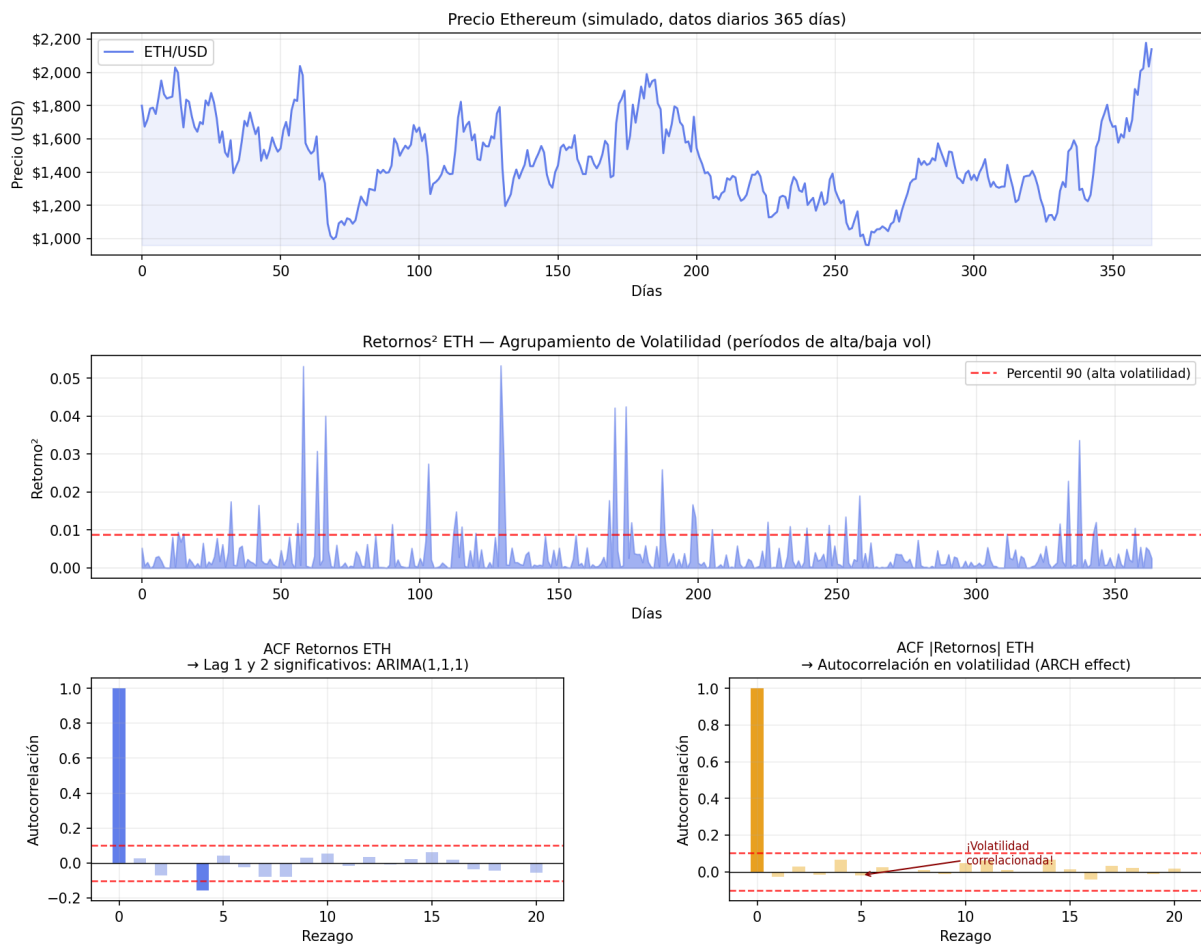


Figure 3: Precio ETH/USD, retornos al cuadrado (agrupamiento de volatilidad), ACF de retornos y ACF de $|\text{retornos}|$ (efecto ARCH).

¿Por qué ARIMA(1,1,1) para Ethereum?

- $d = 1$: Igual que BTC, ETH no es estacionario en niveles. Los log-retornos son estacionarios.
- $p = 1, q = 1$: La ACF muestra un pico significativo en lag 1 y luego decae; la PACF también tiene un pico en lag 1. Este patrón mixto indica un proceso ARMA(1,1).
- **Agrupamiento de volatilidad (Volatility Clustering)**: Los retornos al cuadrado r_t^2 muestran autocorrelación. Esto significa que períodos de alta volatilidad son seguidos por más alta volatilidad. ARIMA modela la media, pero no la varianza.
- **Efecto ARCH**: La ACF de los $|r_t|$ tiene múltiples lags significativos, sugiriendo que ARIMA(1,1,1) debería complementarse con un modelo GARCH(1,1) para la volatilidad.

Diferencia clave: BTC vs ETH

Aunque ambos son criptomonedas, ETH suele tener **mayor volatilidad** y más dinamismos en sus retornos debido a la actividad de contratos inteligentes y DeFi. Por eso ETH necesita un componente MA adicional ($q = 1$) que BTC no requiere.

3.1. Código Python — Ejemplo 2: Ethereum

Listing 2: ARIMA(1,1,1) para Ethereum + diagnóstico de volatilidad

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from statsmodels.tsa.arima.model import ARIMA
5 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
6 from statsmodels.stats.diagnostic import acorr_ljungbox, het_arch
7 import warnings
8 warnings.filterwarnings("ignore")
9
10 # =====
11 # PASO 1: Descargar ETH con yfinance
12 # =====
13 import yfinance as yf
14
15 eth = yf.download("ETH-USD",
16                  start="2022-01-01",
17                  end="2024-12-31",
18                  auto_adjust=True)
19 precio_eth = eth["Close"].dropna()
20 log_eth = np.log(precio_eth)
21 log_ret_eth = log_eth.diff().dropna()
22
23 print(f"ETH - Observaciones: {len(precio_eth)}")
24 print(f"Retorno promedio diario: {log_ret_eth.mean()*100:.3f}%")
25 print(f"Volatilidad diaria (std): {log_ret_eth.std()*100:.2f}%")
26 print(f"Volatilidad anualizada: {log_ret_eth.std()*np.sqrt(365)*100:.1f}%")
27
28 # PASO 2: Visualizar agrupamiento de volatilidad
29 fig, axes = plt.subplots(2, 1, figsize=(13, 7))
30 axes[0].plot(log_ret_eth.values, color="#627EEA",
31             linewidth=0.8, alpha=0.9)
32 axes[0].set_title("Log-Retornos ETH -- Alta volatilidad en cryptos")
```

```

33 axes[0].axhline(0, color="black")
34
35 axes[1].fill_between(range(len(log_ret_eth)),
36                     0, log_ret_eth.values**2,
37                     color="#627EEA", alpha=0.7)
38 axes[1].set_title("Retornos^2 ETH -- Agrupamiento de Volatilidad")
39 axes[1].axhline(log_ret_eth.std()**2 * 4,
40               color="red", linestyle="--",
41               label="4 veces la varianza promedio")
42 axes[1].legend()
43 plt.tight_layout(); plt.show()
44
45 # PASO 3: Prueba de efecto ARCH en los residuales
46 # (verificar si la volatilidad necesita modelarse)
47 test_arch = het_arch(log_ret_eth, nlags=10)
48 print(f"\nPrueba ARCH (LM): estadistico={test_arch[0]:.3f}, "
49       f"p-value={test_arch[1]:.4f}")
50 if test_arch[1] < 0.05:
51     print("Hay efecto ARCH --> Considerar GARCH para la volatilidad")
52 else:
53     print("No hay efecto ARCH significativo")
54
55 # PASO 4: ACF y PACF
56 fig, axes = plt.subplots(2, 1, figsize=(12, 6))
57 plot_acf(log_ret_eth, lags=20, ax=axes[0], color="#627EEA")
58 axes[0].set_title("ACF Retornos ETH: pico en lag 1, luego decae -->
59                  MA(1)")
60 plot_pacf(log_ret_eth, lags=20, ax=axes[1], color="#627EEA")
61 axes[1].set_title("PACF Retornos ETH: pico en lag 1 --> AR(1)")
62 plt.tight_layout(); plt.show()
63 # Patron mixto AR + MA --> ARMA(1,1) --> ARIMA(1,1,1)
64
65 # PASO 5: Ajustar ARIMA(1,1,1)
66 modelo_eth = ARIMA(log_eth, order=(1, 1, 1))
67 res_eth = modelo_eth.fit()
68 print(res_eth.summary())
69 print(f"\nAIC: {res_eth.aic:.2f} | BIC: {res_eth.bic:.2f}")
70
71 # PASO 6: Prueba Ljung-Box sobre residuales
72 residuales = res_eth.resid.dropna()
73 lb_test = acorr_ljungbox(residuales, lags=[5, 10, 20], return_df=True)
74 print("\nPrueba Ljung-Box (p > 0.05 = residuales como ruido blanco):")
75 print(lb_test)
76
77 # PASO 7: Pronostico a 14 dias (2 semanas)
78 fc_eth = res_eth.get_forecast(steps=14)
79 fc_mean = np.exp(fc_eth.predicted_mean)
80 fc_ci = fc_eth.conf_int()
81 fc_lo = np.exp(fc_ci.iloc[:, 0])
82 fc_hi = np.exp(fc_ci.iloc[:, 1])
83
84 plt.figure(figsize=(13, 5))
85 precio_eth[-60:].plot(label="ETH Historico", color="#627EEA",
86                      linewidth=1.5)
87 fc_mean.plot(label="Pronostico 14 dias",
88             color="red", linestyle="--", linewidth=2)

```

```

88 plt.fill_between(fc_ci.index, fc_lo, fc_hi,
89                 alpha=0.2, color="red", label="IC 95%")
90 plt.title("Pronostico ETH/USD -- ARIMA(1,1,1)")
91 plt.ylabel("Precio (USD)")
92 plt.legend()
93 plt.tight_layout(); plt.show()
94
95 # NOTA: Para modelar la VOLATILIDAD de ETH, usar GARCH:
96 # from arch import arch_model
97 # garch = arch_model(log_ret_eth * 100, vol="Garch", p=1, q=1)
98 # res_garch = garch.fit(dispatch="off")
99 # print(res_garch.summary())

```

4. Ejemplo 3: S&P 500 vs IPC México — SARIMA(1,1,1)(1,1,0)₁₂

Ejemplo 3: S&P 500 vs IPC México — SARIMA(1,1,1)(1,1,0)₁₂



Figure 4: S&P 500 e IPC México mensuales, retornos comparados, ACF de cada índice y pronóstico a 20 meses.

¿Por qué SARIMA para índices bursátiles mensuales?

- $d = 1$: Los índices bursátiles tienen tendencia (raíz unitaria). Una diferenciación los hace estacionarios.
- $S = 12$: Con datos mensuales, la ACF muestra picos en los lags 12, 24, 36, confirmando estacionalidad anual (efecto enero, ciclos de reportes trimestrales, temporada de dividendos).
- $D = 1$: Para eliminar la estacionalidad persistente se aplica una diferenciación estacional.
- $P = 1, Q = 0$: La PACF de las diferencias estacionales muestra un pico en lag 12 y luego cae, sugiriendo un componente AR estacional de orden 1.
- **IPC México vs S&P 500**: Ambos comparten la estructura SARIMA, pero el IPC tiene mayor volatilidad y dependencia del tipo de cambio USD/MXN y los precios del petróleo.

Relación entre S&P 500 e IPC México

Los mercados bursátiles mexicano y americano están **altamente correlacionados** (correlación histórica ≈ 0.70 – 0.85). Cuando el S&P 500 cae fuerte, el IPC generalmente también cae. Esto ocurre porque:

- México exporta más del 80% de sus bienes a EE.UU.
- Los inversores institucionales gestionan carteras latinoamericanas de forma conjunta
- El tipo de cambio USD/MXN amplifica los movimientos (riesgo-país)

Para capturar esta relación se puede usar **SARIMAX** (con el S&P 500 como variable exógena en el modelo del IPC).

4.1. Código Python — Ejemplo 3: S&P 500 e IPC México

Listing 3: SARIMA para S&P 500 e IPC México con yfinance — datos mensuales

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from statsmodels.tsa.statespace.sarimax import SARIMAX
5 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
6 from statsmodels.tsa.seasonal import seasonal_decompose
7 import warnings
8 warnings.filterwarnings("ignore")
9
10 # =====
11 # PASO 1: Descargar datos mensuales con yfinance
12 # =====
13 import yfinance as yf
14
15 # S&P 500
16 sp500_raw = yf.download("^GSPC",
17                        start="2014-01-01",
18                        end="2024-12-31",
19                        interval="1mo",
20                        auto_adjust=True)
21 sp500 = sp500_raw["Close"].dropna()

```

```

22
23 # IPC Mexico (ticker en Yahoo Finance)
24 ipc_raw = yf.download("^MXX",
25                       start="2014-01-01",
26                       end="2024-12-31",
27                       interval="1mo",
28                       auto_adjust=True)
29 ipc = ipc_raw["Close"].dropna()
30
31 # Alinear los indices de fechas
32 idx_comun = sp500.index.intersection(ipc.index)
33 sp500 = sp500[idx_comun]
34 ipc = ipc[idx_comun]
35 print(f"Observaciones mensuales: {len(sp500)}")
36
37 # PASO 2: Descomposicion estacional
38 fig, axes = plt.subplots(2, 4, figsize=(16, 7))
39 for col, (serie, nombre, color) in enumerate(zip(
40     [sp500, ipc], ["S&P 500", "IPC Mexico"],
41     ["#003087", "#006847"])):
42     descomp = seasonal_decompose(serie, model="multiplicative",
43                                 period=12)
44     descomp.observed.plot(ax=axes[0, col*2], color=color,
45                          title=f"{nombre} - Original")
46     descomp.trend.plot(ax=axes[0, col*2+1], color=color,
47                       title="Tendencia")
48     descomp.seasonal.plot(ax=axes[1, col*2], color="orange",
49                          title="Estacionalidad")
50     descomp.resid.plot(ax=axes[1, col*2+1], color="gray",
51                       title="Residual")
52 plt.suptitle("Descomposicion Estacional: S&P500 vs IPC", fontsize=13)
53 plt.tight_layout(); plt.show()
54
55 # PASO 3: ACF para confirmar estacionalidad S=12
56 fig, axes = plt.subplots(2, 2, figsize=(14, 8))
57 for row, (serie, nombre) in enumerate(zip([sp500, ipc],
58                                           ["S&P 500", "IPC Mexico"])):
59     :
60     diff_s = serie.diff().dropna()
61     plot_acf(diff_s, lags=36, ax=axes[row, 0])
62     axes[row, 0].set_title(f"ACF 1ra Diferencia - {nombre}")
63     axes[row, 0].set_xlabel("Rezago (meses)")
64
65     plot_pacf(diff_s, lags=36, ax=axes[row, 1])
66     axes[row, 1].set_title(f"PACF 1ra Diferencia - {nombre}")
67     axes[row, 1].set_xlabel("Rezago (meses)")
68
69     # Marcar lags estacionales
70     for ax in axes[row]:
71         for sl in [12, 24, 36]:
72             ax.axvline(sl, color="orange", linestyle=":",
73                       alpha=0.6, linewidth=1.5)
74 plt.tight_layout(); plt.show()
75 # Picos en 12, 24 --> estacionalidad anual, S=12, D=1 necesario
76
77 # PASO 4: Ajustar SARIMA(1,1,1)(1,1,0)_12 para cada indice
78 resultados_modelos = {}
79 for serie, nombre in zip([sp500, ipc], ["SP500", "IPC"]):

```

```

79     modelo = SARIMAX(serie,
80                     order=(1, 1, 1),
81                     seasonal_order=(1, 1, 0, 12),
82                     enforce_stationarity=False,
83                     enforce_invertibility=False)
84     res = modelo.fit(dispatch=False)
85     resultados_modelos[nombre] = res
86     print(f"\n--- {nombre} ---")
87     print(f"AIC: {res.aic:.2f} | BIC: {res.bic:.2f}")
88     print(res.summary().tables[1])
89
90 # PASO 5: Comparar varios modelos (solo para SP500)
91 print("\nGrid search SARIMA para S&P 500:")
92 candidatos = [
93     (1,1,1,1,1,0), (1,1,0,1,1,0),
94     (1,1,1,1,1,1), (2,1,1,1,1,0),
95     (1,1,1,0,1,1)
96 ]
97 for cfg in candidatos:
98     try:
99         m = SARIMAX(sp500,
100                    order=cfg[:3],
101                    seasonal_order=(cfg[3:], 12),
102                    enforce_stationarity=False,
103                    enforce_invertibility=False).fit(dispatch=False)
104         print(f"SARIMA{cfg[:3]}{cfg[3:]}12 | "
105               f"AIC={m.aic:.1f} | BIC={m.bic:.1f}")
106     except:
107         pass
108
109 # PASO 6: Pronostico a 18 meses (1.5 años)
110 fig, ax = plt.subplots(figsize=(14, 5))
111 ax2 = ax.twinx()
112
113 for res, serie, nombre, color, ax_use in zip(
114     [resultados_modelos["SP500"], resultados_modelos["IPC"]],
115     [sp500, ipc], ["S&P 500", "IPC Mexico"],
116     ["#003087", "#006847"], [ax, ax2]):
117     fc = res.get_forecast(steps=18)
118     fc_mean = fc.predicted_mean
119     fc_ci = fc.conf_int()
120     serie[-24:].plot(ax=ax_use, color=color, linewidth=1.5,
121                     label=f"{nombre} historico")
122     fc_mean.plot(ax=ax_use, color=color, linewidth=2.5,
123                 linestyle="--", label=f"{nombre} pronostico")
124     ax_use.fill_between(fc_ci.index,
125                        fc_ci.iloc[:, 0], fc_ci.iloc[:, 1],
126                        alpha=0.15, color=color)
127
128 ax.set_ylabel("S&P 500 (puntos)", color="#003087")
129 ax2.set_ylabel("IPC Mexico (puntos)", color="#006847")
130 ax.set_title("Pronostico 18 meses: S&P 500 vs IPC Mexico "
131             "- SARIMA(1,1,1)(1,1,0)_12")
132 lines1, labs1 = ax.get_legend_handles_labels()
133 lines2, labs2 = ax2.get_legend_handles_labels()
134 ax.legend(lines1+lines2, labs1+labs2, fontsize=9)
135 plt.tight_layout(); plt.show()
136

```

```

137 # PASO 7: SARIMAX -- usar S&P500 como variable exogena del IPC
138 modelo_sarimax = SARIMAX(ipc,
139                             exog=sp500,
140                             order=(1, 1, 1),
141                             seasonal_order=(1, 1, 0, 12),
142                             enforce_stationarity=False,
143                             enforce_invertibility=False)
144 res_sarimax = modelo_sarimax.fit(dispatch=False)
145 print(f"\nSARIMAX (con S&P500 como exogena):")
146 print(f"AIC: {res_sarimax.aic:.2f} | BIC: {res_sarimax.bic:.2f}")
147 print(res_sarimax.summary().tables[1])

```

5. Resumen Comparativo de los 3 Ejemplos

Activo	Frecuencia	Modelo	Razón principal	Característica especial
Bitcoin BTC	Diaria	ARIMA(1,1,0)	PACF corta en lag 1	Colas pesadas (t -Student)
Ethereum ETH	Diaria	ARIMA(1,1,1)	ACF y PACF con lag 1	Agrupamiento de volatilidad (GARCH)
S&P 500	Mensual	SARIMA(1, 1, 1)(1, 0, 0, 12)	Picco en lag 12	Ciclo anual de mercados
IPC México	Mensual	SARIMA(1, 1, 1)(1, 0, 0, 12)	Igual que S&P 500	Correlación con USD/MXN

Limitaciones importantes del modelo ARIMA en finanzas

1. **No modela la volatilidad:** ARIMA captura la media condicional, no la varianza. Para volatilidad usa **GARCH(1,1)**.
2. **Linealidad:** ARIMA asume relaciones lineales. Los mercados crypto tienen dinámicas no lineales.
3. **Horizonte de pronóstico:** En crypto, los pronósticos más allá de 7–14 días tienen intervalos de confianza muy amplios.
4. **Eventos extremos:** Los crashes (COVID marzo 2020, FTX noviembre 2022) no son captados por ARIMA.
5. **No considera sentimiento:** Noticias, redes sociales y regulación afectan los precios crypto fuertemente.

— Fin del Documento —